CHƯƠNG TRÌNH FORECASTING CHO CHUỖI CUNG ỨNG

MANUAL PYTHON CƠ BẢN

Đọc hiểu thành phần code và kiểm chứng kết quả

Dành cho người mới bắt đầu — kèm hướng dẫn Vibe Coding an toàn với AI

Tài liệu bổ sung

Thư Nguyễn

MSc Logistics and Supply Chain Analytics — University of Southampton, UK

thu.minh090997@gmail.com

Biên soạn theo chương trình 10 buổi
Cập nhật: Ngày 13 tháng 9 năm 2026

Mục lục

1 Python là gì, và vì sao khóa học dùng nó **1**

2 Cách đọc một file code Python **1**

2.1 Dòng chú thích (comment) 1

2.2 Biến (variable) 1

2.3 Danh sách (list) và bảng dữ liệu (DataFrame) 2

3 Các thư viện (library) dùng trong khóa học **2**

4 Đọc hiểu một đoạn code thật từ khóa học **2**

5 Vòng lặp (for): thực hiện lại một thao tác nhiều lần **4**

5.1 Bảng dò theo dõi biến (variable trace): 6 vòng lặp đầu tiên 5

6 Hàm (function): đóng gói một công thức để tái sử dụng **7**

7 Chạy code và kiểm chứng (validate) kết quả **7**

7.1 Cách chạy code 7

7.2 Ba cách kiểm chứng kết quả 8

8 Lỗi thường gặp khi mới học Python **8**

9 Dùng AI hỗ trợ viết code (“vibe coding”) mà không làm lộ dữ liệu công ty **9**

9.1 Nguyên tắc cốt lõi: tự quan sát hình dạng dữ liệu trước, hỏi AI sau 9

9.2 Script tổng quát: áp dụng cho BẤT KỲ file Excel nào 9

9.3 Quy trình “vibe coding” 3 bước an toàn 10

10 Bảng tra nhanh — các hàm thường gặp trong khóa học **11**

11 Áp dụng thực tế: chạy thử trên bộ dữ liệu THẬT khác (Indonesia) **11**

12 Đọc hiểu code mở rộng: Quadratic Trend **13**

13 Đọc hiểu code mở rộng: hồi quy đa biến với yếu tố ngoại sinh liên tục **14**

14 Đọc hiểu code nâng cao: hàm Croston cho nhu cầu rời rạc **16**

14.1 Code đầy đủ, đọc từng khối 16

14.2 Kết quả thật trên SKU X12 18

15 Đọc hiểu code kiểm tra thật: Train/Test Split **19**

15.1 Vì sao cần đọc kỹ đoạn code này 19

15.2 Điểm cần đặc biệt cẩn thận: rò rỉ dữ liệu khi chọn tham số ARIMA 20

16 Công cụ thực hành: Forecasting Lab (ứng dụng Streamlit) **21**

17 Những dòng code này đang chạy ở đâu trong phần mềm thương mại? **22**

Code & Demo **23**

1 Python là gì, và vì sao khóa học dùng nó

Python là một ngôn ngữ lập trình được thiết kế để dễ đọc — cú pháp gần với tiếng Anh thông thường hơn so với nhiều ngôn ngữ lập trình khác. Người học không cần biết lập trình để hiểu các khái niệm forecast trong khóa học, nhưng biết đọc code Python sẽ giúp tự kiểm chứng được kết quả, thay vì phải tin tưởng vào một con số do người khác cung cấp.

Trong khóa học, Python được sử dụng cho hai mục đích chính: (1) tính toán các mô hình forecast mà Excel không thực hiện trực tiếp được, ví dụ ARIMA hoặc Regression với nhiều biến giải thích; và (2) trực quan hoá dữ liệu bằng biểu đồ. Toàn bộ mã nguồn trong khóa học đã được viết sẵn — nhiệm vụ của người học là đọc hiểu, không phải tự viết từ đầu.

2 Cách đọc một file code Python

Một file code Python (đuôi mở rộng .py) là một dãy các câu lệnh được thực thi tuần tự từ trên xuống dưới — tương tự một công thức nấu ăn, bước 1 hoàn tất thì mới chuyển sang bước 2.

2.1 Dòng chú thích (comment)

Bất kỳ dòng nào bắt đầu bằng dấu # đều là ghi chú — Python bỏ qua hoàn toàn, không thực thi. Đây là lời giải thích của người viết code dành cho người đọc.

```
# Day la ghi chu - Python se bo qua dong nay
x = 5  # Day cung la ghi chu, dung sau lenh
```

2.2 Biến (variable)

Biến là một vùng nhớ có tên, dùng để lưu trữ giá trị. Dấu bằng (=) trong Python có nghĩa là gán giá trị, khác với ký hiệu bằng nhau trong toán học.

```
ten_san_pham = "Noi com dien"    # bien kieu chu (string)
so_luong = 758                     # bien kieu so nguyen (int)
ty_le = 0.35                       # bien kieu so thap phan (float)
```

Cách kiểm tra

Nếu gặp một biến lạ trong code, hãy tìm dòng đầu tiên nó xuất hiện (thường có dấu bằng) để biết giá trị được gán là gì.

2.3 Danh sách (list) và bảng dữ liệu (DataFrame)

List là một dãy giá trị đặt trong ngoặc vuông, ví dụ danh sách doanh số của 3 tháng. DataFrame — đến từ thư viện pandas — là một bảng dữ liệu hai chiều, tương tự một bảng tính Excel: có hàng và cột, có thể lọc và tính toán trên từng cột.

```
doanh_so = [758, 430, 462]          # list - giong 1 cot trong Excel

import pandas as pd
df = pd.DataFrame({
    "Thang": [1, 2, 3],
    "Doanh_so": [758, 430, 462]
})                                    # DataFrame - giong ca 1 bang
    Excel
```

3 Các thư viện (library) dùng trong khóa học

Thư viện là tập hợp công cụ có sẵn do người khác viết, giúp người dùng không phải tự viết lại từ đầu. Dòng lệnh import ở đầu mỗi file code có tác dụng nạp các công cụ này vào chương trình.

Thư viện	Dùng để làm gì	Ví dụ trong khóa học
pandas (pd)	Xử lý bảng dữ liệu, tương tự Excel	Đọc file CSV, lọc theo tháng, tính trung bình theo nhóm
numpy (np)	Tính toán số học trên mảng số lớn	Căn bậc hai cho Safety Stock, trung bình, độ lệch chuẩn
matplotlib	Vẽ biểu đồ	Toàn bộ biểu đồ trong slide bài giảng
statsmodels	Các mô hình thống kê chuyên sâu	SES, Holt-Winters, ARIMA, Regression (Buổi 3-5)
sklearn	Công thức đo sai số có sẵn	mean_absolute_error, mean_squared_error (Buổi 2)

4 Đọc hiểu một đoạn code thật từ khóa học

Phần này minh hoạ cách đọc từng dòng của một đoạn code thực tế đã sử dụng ở Buổi 3, tính SES (Simple Exponential Smoothing):

```
from statsmodels.tsa.api import SimpleExpSmoothing

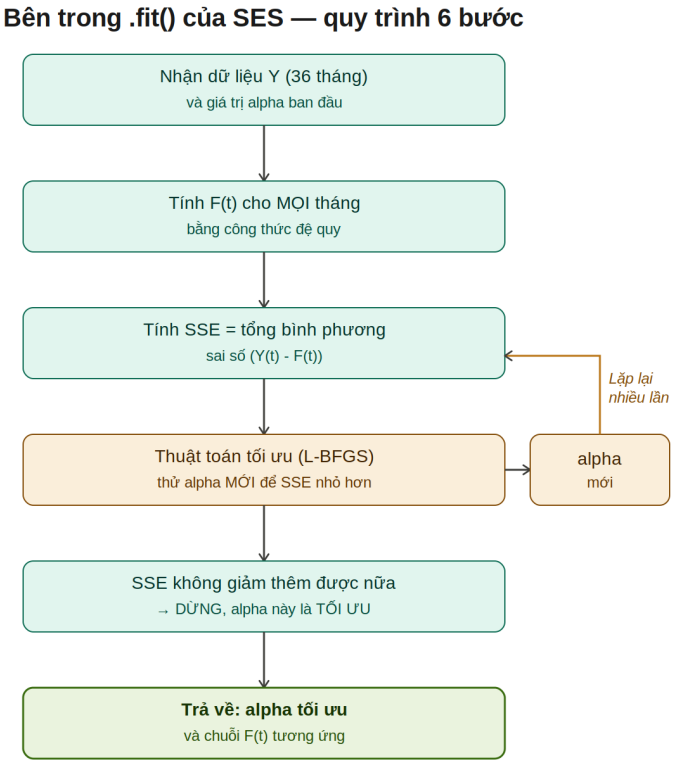
# Buoc 1: dung mo hinh SES tren du lieu, de Python tu tim alpha toi
    uu
ses_fit = SimpleExpSmoothing(s_beptu).fit()
```

```
# Buoc 2: lay gia tri alpha ma Python da tim ra
alpha_opt = ses_fit.model.params["smoothing_level"]

# Buoc 3: in ket qua ra man hinh de xem
print(f"Alpha toi uu = {alpha_opt:.4f}")
```

- **Dòng 1:** nạp công cụ SimpleExpSmoothing từ thư viện statsmodels.
- **Dòng 4:** gọi hàm SimpleExpSmoothing(), truyền vào dữ liệu s_beptu (chuỗi số của SKU Bếp từ đôi), sau đó gọi .fit() để yêu cầu Python tự động tìm tham số tối ưu — tương đương thao tác Solver trong Excel.

Dòng `.fit()` trông đơn giản nhưng thực chất "giấu"bên trong nó cả một quy trình lặp nhiều bước — người mới học không cần tự viết lại quy trình này, nhưng hiểu được nó giúp đọc code tự tin hơn, không còn coi `.fit()` là "hộp đen bí ẩn":

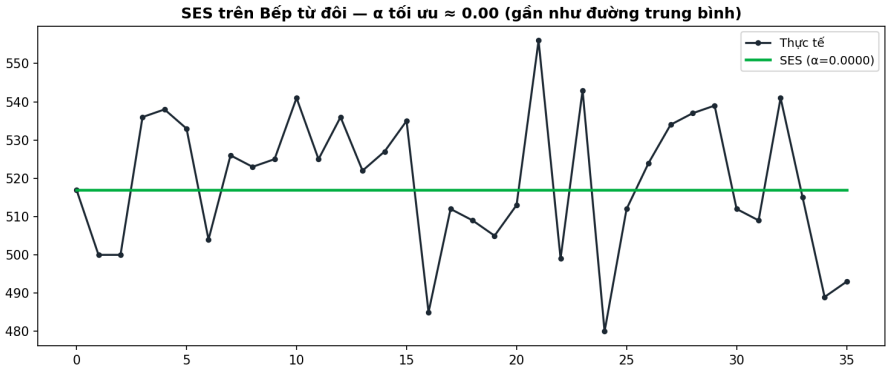


- **Dòng 7:** kết quả tính toán được lưu trong biến ses_fit; cần truy cập vào thuộc tính bên trong để lấy giá trị alpha cụ thể.
- **Dòng 10:** lệnh print() là cách duy nhất để hiển thị kết quả trên màn hình — nếu thiếu dòng này, Python vẫn tính đúng nhưng người dùng sẽ không nhìn thấy kết quả.

Cách kiểm tra

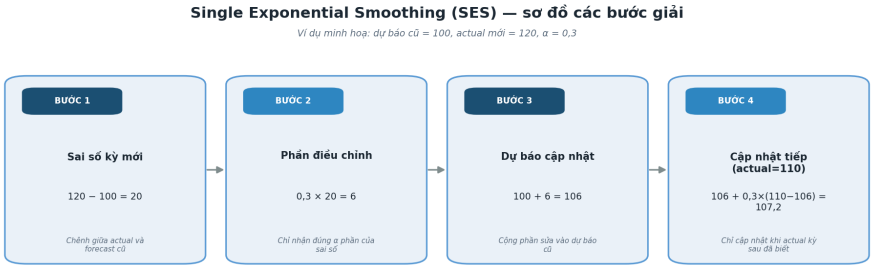
Luôn tìm các dòng print() trong code — đó là nơi kết quả được hiển thị để đối chiếu với đáp án.

Chạy đoạn code trên với dữ liệu thật của Bếp từ đôi cho ra alpha tối ưu gần bằng 0 — và đây chính là hình ảnh mà đoạn code đó vẽ ra:



Đường dự báo gần như phẳng vì alpha thấp khiến mô hình "tin vào mức nền"hơn là chạy theo từng quan sát mới — đúng như giải thích ở Lecture Notes Chương 3. Đây là cách hữu ích để tự kiểm tra: nếu code chạy xong mà đồ thị KHÔNG giống mô tả lý thuyết, nhiều khả năng có lỗi ở đâu đó trong code hoặc dữ liệu đầu vào.

Để đối chiếu công thức tay với đoạn code vừa đọc, sơ đồ dưới đây tóm tắt lại đúng trình tự các bước tính SES trên một ví dụ minh hoạ (xem thêm Lecture Notes Chương 3):

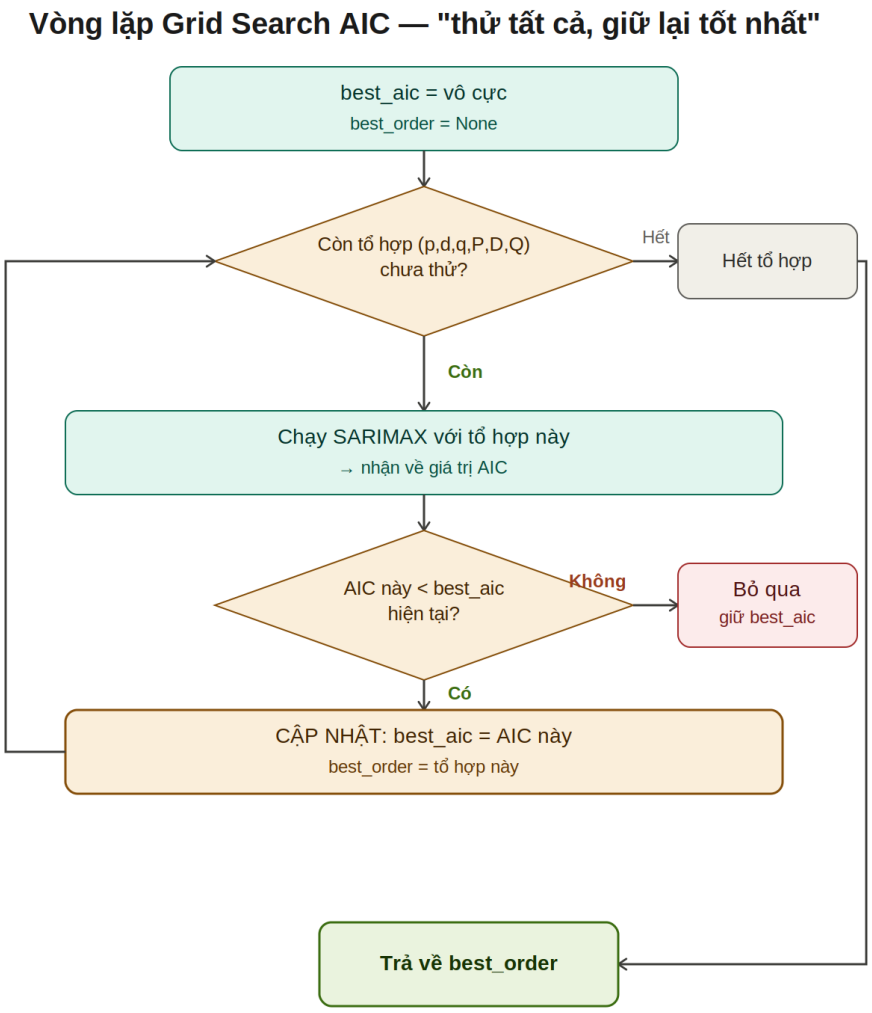


5 Vòng lặp (for): thực hiện lại một thao tác nhiều lần

Nhiều đoạn code trong khóa học — đặc biệt ở Buổi 4, khi tìm tham số ARIMA tối ưu — sử dụng vòng lặp: yêu cầu Python lặp lại một khối lệnh nhiều lần thay vì phải viết tay hàng chục lần.

```
# Thu tat ca cac gia tri order tu 1 danh sach, tim order co AIC nho
    nhat
best_aic = 999999
for order in danh_sach_order:
    ket_qua = chay_mo_hinh(order)
    if ket_qua.aic < best_aic:
        best_aic = ket_qua.aic
        best_order = order
```

Diễn giải bằng lời: với mỗi order trong danh sách, chạy mô hình; nếu AIC của lần chạy này nhỏ hơn AIC nhỏ nhất đã tìm được trước đó, cập nhật lại AIC nhỏ nhất và ghi nhớ order tương ứng. Đây chính là cách con người sẽ làm nếu so sánh thủ công hàng chục kết quả — Python chỉ thực hiện nhanh hơn.



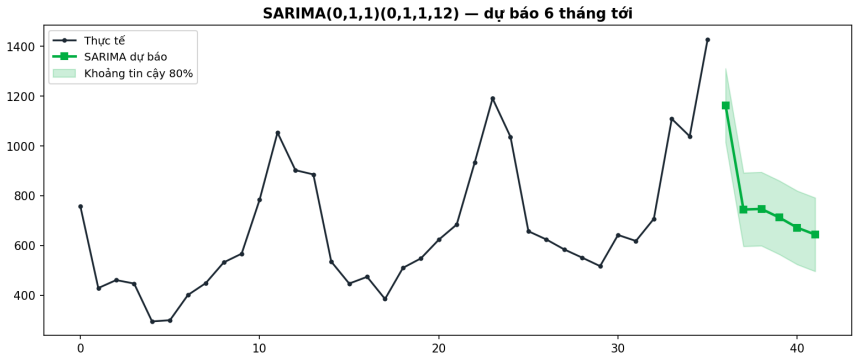
5.1 Bảng dò theo dõi biến (variable trace): 6 vòng lặp đầu tiên

Cách tốt nhất để hiểu một vòng lặp là "chạy tay"nó trên giấy — theo dõi giá trị của từng biến sau MỖI lần lặp, giống hệt cách phần mềm thực hiện bên trong:

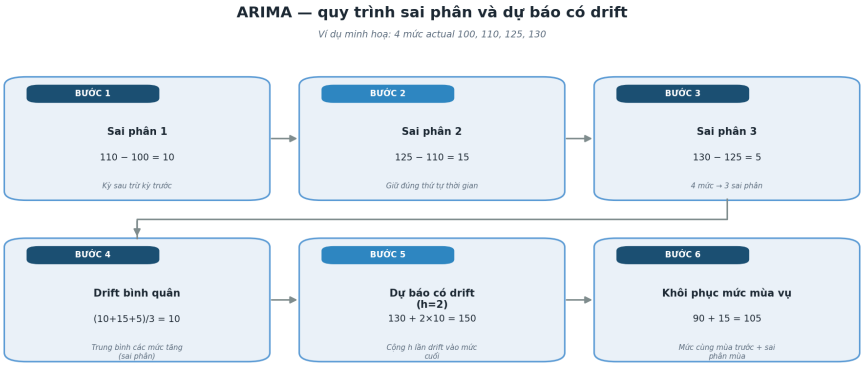
order đang thử	AIC vừa tính	best_aic SAU dòng này	best_order SAU dòng này
(0,1,0)(0,1,0)	291,7	291,7	(0,1,0)(0,1,0)
(0,1,0)(0,1,1)	133,8	**133,8** (cập nhật)	(0,1,0)(0,1,1)
(0,1,1)(0,1,0)	265,9	133,8 (giữ nguyên)	(0,1,0)(0,1,1)
(0,1,1)(0,1,1)	117,1	**117,1** (cập nhật)	(0,1,1)(0,1,1)
(1,1,0)(0,1,1)	130,8	117,1 (giữ nguyên)	(0,1,1)(0,1,1)
(1,1,1)(0,1,1)	118,8	117,1 (giữ nguyên)	(0,1,1)(0,1,1)

Điểm mấu chốt cần quan sát: `best_aic` chỉ THAY ĐỔI ở hai dòng được tô đậm — những lần còn lại, khối lệnh trong `if` không chạy vì điều kiện sai, biến giữ nguyên giá trị cũ. Đây chính là cách một vòng lặp "nhớ"được kết quả tốt nhất qua nhiều lần thử, mà không cần lưu lại TẤT CẢ các kết quả trung gian.

Áp dụng đầy đủ cho Nồi cơm điện (72 tổ hợp, không chỉ 6 dòng minh hoạ ở trên), vòng lặp chọn ra order (0,1,1)(0,1,1,12) — và đây là dự báo mà order đó tạo ra:



Vòng lặp grid search vừa đọc chỉ đảm nhiệm việc CHỌN tham số (p,d,q) — sơ đồ dưới đây tóm tắt lại phần TÍNH TOÁN phía sau mỗi tham số đó: các bước kiểm tra dừng, sai phân và dự báo có drift của ARIMA:



6 Hàm (function): đóng gói một công thức để tái sử dụng

Khi một công thức cần sử dụng lặp lại nhiều lần — ví dụ tính năm chỉ số sai số cho nhiều SKU khác nhau — công thức đó được đóng gói thành một hàm, tương tự việc tạo một công thức Excel tùy chỉnh.

```
def tinh_mape(thuc_te, du_bao):
    sai_so = thuc_te - du_bao
    mape = (abs(sai_so) / thuc_te).mean() * 100
    return mape

# Goi ham cho 2 SKU khac nhau - khong can viet lai cong thuc
mape_noicom = tinh_mape(y_noicom, f_noicom)
mape_beptu = tinh_mape(y_beptu, f_beptu)
```

- def là từ khoá bắt đầu định nghĩa một hàm mới; tên hàm (tinh_mape) do người viết code tự đặt.
- Phần trong ngoặc (thuc_te, du_bao) là đầu vào — tương tự hai tham chiếu ô trong công thức Excel dạng =MAPE(A1:A10, B1:B10).
- Dòng return là đầu ra — giá trị hàm trả về sau khi tính toán hoàn tất.

7 Chạy code và kiểm chứng (validate) kết quả

7.1 Cách chạy code

Mỗi file .py trong khóa học được chạy bằng lệnh sau, gõ trong cửa sổ dòng lệnh (Terminal hoặc Command Prompt), tại đúng thư mục chứa file:

python3 Buoi3_01_BieuDo.py

Sau khi chạy, Python in ra màn hình toàn bộ nội dung của các lệnh print() có trong file, đồng thời lưu các biểu đồ thành file ảnh định dạng .png vào thư mục imgs/.

7.2 Ba cách kiểm chứng kết quả

- **Cách 1 — So khớp với số đã biết:** mọi con số quan trọng trong khóa học (MAPE, hệ số R-squared, Safety Stock...) đều đã được nêu trong slide bài giảng và file đáp án. Nếu chạy lại code và thu được đúng những con số đó, code chạy đúng.
- **Cách 2 — Kiểm tra tính hợp lý (sanity check):** giá trị MAPE có nằm trong khoảng 0-100% không? Giá trị alpha có nằm trong khoảng 0-1 không? Nếu kết quả là số âm hoặc số lớn bất thường, khả năng cao có lỗi ở một bước nào đó.
- **Cách 3 — Đối chiếu Excel và Python:** phần lớn công thức trong khóa học có cả phiên bản Excel và Python song song, ví dụ Solver trong Excel và hàm SimpleExpSmoothing trong Python đều tìm alpha tối ưu. Nếu hai kết quả khớp nhau (chênh lệch không đáng kể do làm tròn số), có thể tin tưởng cả hai.

Ghi chú

Toàn bộ dữ liệu trong khóa học sử dụng chung một hạt giống ngẫu nhiên — numpy.random.default_rng(42) — nên bất kỳ ai chạy lại code trên máy tính đều thu được đúng những con số giống hệt nhau. Đây gọi là tính tái lập (reproducibility), một yêu cầu quan trọng trong khoa học dữ liệu.

8 Lỗi thường gặp khi mới học Python

Lỗi ModuleNotFoundError

Thông báo No module named 'pandas' nghĩa là máy chưa cài thư viện đó. Khắc phục bằng lệnh: pip install pandas –break-system-packages

Lỗi IndentationError

Python rất nhạy cảm với khoảng trắng ở đầu dòng (thụt lề). Các dòng nằm trong cùng một khối lệnh, ví dụ bên trong vòng lặp for, phải được thụt lề giống hệt nhau.

Lỗi NameError

Thông báo name 'x' is not defined nghĩa là đang sử dụng một biến chưa được gán giá trị trước đó — có thể do quên chạy dòng gán, hoặc gõ sai tên biến.

Kết quả không khớp đáp án

Cần kiểm tra lại: có đang sử dụng đúng seed=42 hay không, và có đang chạy đúng phiên bản file gốc, không bị chỉnh sửa nhầm dòng nào hay không.

9 Dùng AI hỗ trợ viết code ("vibe coding") mà không làm lộ dữ liệu công ty

Sau khóa học, nhiều người sẽ muốn dùng công cụ AI (ChatGPT, Claude, Copilot...) để viết nhanh code Python xử lý SKU thật của công ty mình. Đây là cách làm hiệu quả — nhưng có một rủi ro cần lưu ý: dán trực tiếp bảng số liệu bán hàng thật vào cửa sổ chat với AI có thể vi phạm chính sách bảo mật dữ liệu của công ty, đặc biệt với các SKU chiến lược hoặc dữ liệu doanh thu nhạy cảm. Phần này trình bày một quy trình làm việc tránh được rủi ro đó, đồng thời khớp với nguyên tắc "lưu code và dữ liệu cùng một thư mục, dùng đường dẫn tương đối" đã áp dụng xuyên suốt bộ code của khóa học.

9.1 Nguyên tắc cốt lõi: tự quan sát hình dạng dữ liệu trước, hỏi AI sau

Thay vì dán số liệu thật vào cuộc trò chuyện với AI, hãy tự chạy một đoạn code TỔNG QUÁT (không chứa số liệu cụ thể nào) để quan sát HÌNH DẠNG dữ liệu của mình trước — vẽ time plot, phân rã (decompose) thành xu hướng/mùa vụ/phần dư, giống hệt kỹ thuật đã học ở Buổi 1. Sau khi tự nhìn thấy hình dạng, câu hỏi gửi cho AI chỉ cần mô tả ĐẶC ĐIỂM ("SKU của tôi có xu hướng tăng nhẹ và mùa vụ rõ rệt chu kỳ 12 tháng, %kỳ=0 khoảng 20%") — không cần kèm bất kỳ con số doanh số thật nào. AI vẫn có thể tư vấn phương pháp phù hợp và viết code mẫu chính xác không kém, vì bản thân các công thức forecast (SES, Holt-Winters, ARIMA...) không phụ thuộc vào giá trị số liệu cụ thể, chỉ phụ thuộc vào ĐẶC ĐIỂM THỐNG KÊ của dữ liệu.

9.2 Script tổng quát: áp dụng cho BẤT KỲ file Excel nào

Đoạn code dưới đây là một mẫu "đọc và quan sát" tổng quát — không hardcode tên SKU hay số liệu nào, chỉ cần đặt cùng thư mục với file Excel của bạn (đúng nguyên tắc lưu trữ đã học) và đổi tên file ở dòng đầu tiên:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
import os

# CHI CAN DOI TEN FILE O DAY - script con lai dung cho MOI file Excel
TEN_FILE = "du_lieu_cua_toi.xlsx"
TEN_SHEET = 0           # 0 = sheet dau tien, hoac doi thanh ten sheet
COT_NGAY = 0            # cot dau tien la ngay/thang
COT_SO_LUONG = 1        # cot thu hai la san luong
```

```
os.makedirs("imgs", exist_ok=True)
df = pd.read_excel(TEN_FILE, sheet_name=TEN_SHEET)
ser = pd.Series(df.iloc[:, COT_SO_LUONG].values,
                index=pd.to_datetime(df.iloc[:, COT_NGAY]))
ser = ser.asfreq("MS")  # doi "MS" thanh "W"/"D" neu la du lieu tuan/
    ngay

# Buoc 1: nhin tong quan
ser.plot(title="Time plot", figsize=(10, 4))
plt.savefig("imgs/quan_sat_time_plot.png", dpi=150, bbox_inches="
    tight")
plt.close()

# Buoc 2: phan ra xu huong / mua vu / phan du (nhu Buoi 1)
result = seasonal_decompose(ser, model="multiplicative", period=12)
result.plot()
plt.savefig("imgs/quan_sat_decompose.png", dpi=150, bbox_inches="
    tight")
plt.close()

# Buoc 3: in CHI SO THONG KE (khong phai so lieu goc) de mo ta cho AI
cv = ser.std() / ser.mean()
pct_zero = (ser == 0).mean()
print(f"CV = {cv:.3f}")
print(f"%ky=0 = {pct_zero*100:.1f}%")
print("-> Dung 2 con so tren (KHONG phai du lieu goc) de mo ta dac
    diem cho AI.")
```

Điều quan trọng nhất

Hai file ảnh (quan_sat_time_plot.png, quan_sat_decompose.png) và hai chỉ số CV/%kỳ=0 in ra màn hình là TẤT CẢ những gì bạn cần để tự đọc hiểu dữ liệu hoặc mô tả cho AI — không một dòng nào trong đoạn code trên chứa số liệu bán hàng thật, vì mọi phép tính đều tham chiếu qua tên biến (ser), không in trực tiếp bảng dữ liệu gốc.

9.3 Quy trình "vibe coding" 3 bước an toàn

- **Bước 1 — Tự quan sát:** chạy script tổng quát ở trên trên máy của bạn, với file Excel thật. Không dòng nào cần rời khỏi máy bạn.
- **Bước 2 — Mô tả đặc điểm, không mô tả số liệu:** hỏi AI dựa trên hình dạng đã quan sát được, ví dụ "tôi có 1 SKU với CV=0.4, %kỳ=0=0%, có mùa vụ rõ chu kỳ 12 — nên thử phương pháp nào theo framework 6 bước đã học?" thay vì dán nguyên bảng 36 dòng số liệu.
- **Bước 3 — Nhận code mẫu, tự chạy trên dữ liệu thật:** AI sẽ đưa code TỔNG QUÁT (biến tên file, tên cột) — bạn tự điền tên file thật của mình và chạy tại máy, không cần gửi lại kết quả cho AI để xác nhận (đối chiếu bằng file đáp án của khóa học hoặc quy tắc đã học).

Lưu ý

Nếu công ty có chính sách cho phép chia sẻ dữ liệu với công cụ AI được duyệt, quy trình trên vẫn hữu ích để tiết kiệm thời gian — nhưng khi KHÔNG chắc chắn về chính sách, mặc định coi mọi số liệu doanh số là nhạy cảm và áp dụng quy trình 3 bước này.

10 Bảng tra nhanh — các hàm thường gặp trong khóa học

Lệnh	Ý nghĩa
`pd.read_csv(...)`	Đọc một file CSV thành bảng DataFrame
`df.mean(), df.std()`	Tính trung bình, độ lệch chuẩn của một cột
`np.sqrt(x)`	Tính căn bậc hai, dùng trong công thức Safety Stock
`plt.plot(x, y)`	Vẽ một đường trên biểu đồ
`model.fit()`	Yêu cầu Python tự tìm tham số tối ưu cho một mô hình thống kê
`model.predict(...)`	Dùng mô hình đã fit để dự báo giá trị mới
`print(f"...{x:.2f}")`	In ra màn hình, làm tròn số x đến hai chữ số thập phân

11 Áp dụng thực tế: chạy thử trên bộ dữ liệu THẬT khác (Indonesia)

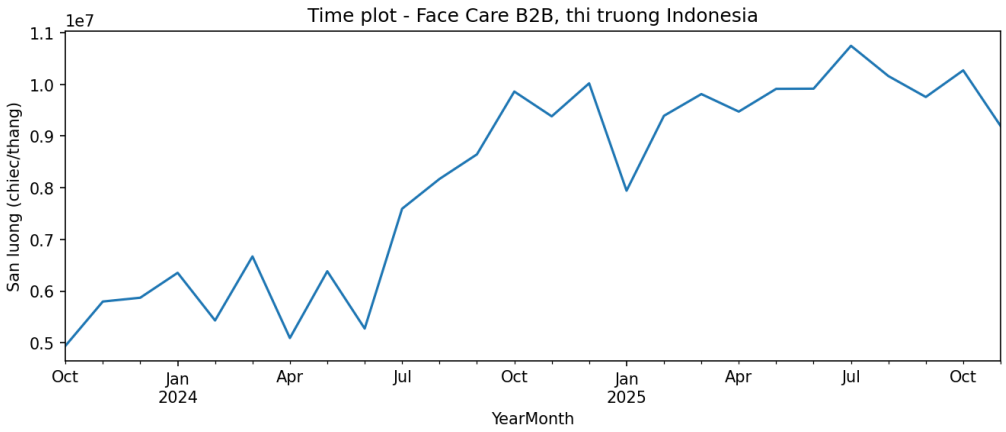
Để chứng minh script tổng quát ở Mục 9.2 thực sự KHÔNG hardcode bất kỳ điều gì, phần này chạy nguyên script đó (chỉ đổi tên file) trên một bộ dữ liệu THẬT hoàn toàn khác — không phải ERP mô phỏng của khóa học, mà là dữ liệu bán hàng ngành hàng chăm sóc da mặt (Face Care), thị trường Indonesia, 27 tháng (09/2023–11/2025), đã ẩn danh ở mức tổng hợp. Nếu code chạy đúng ở đây mà không cần sửa một dòng nào ngoài tên file, đó là bằng chứng code được viết ĐÚNG CÁCH (tổng quát, không hardcode).

```
TEN_FILE = "face_care_indonesia.csv"      # doi ten file, khong doi gi
    khac trong script
df = pd.read_csv(TEN_FILE)
ser = pd.Series(df["DemandInPCS"].values,
                index=pd.to_datetime(df["YearMonth"] + "-01"))
ser = ser.asfreq("MS")

# Buoc 1: nhin tong quan
ser.plot(title="Time plot")
plt.savefig("imgs/quan_sat_time_plot.png", dpi=150, bbox_inches="
    tight")

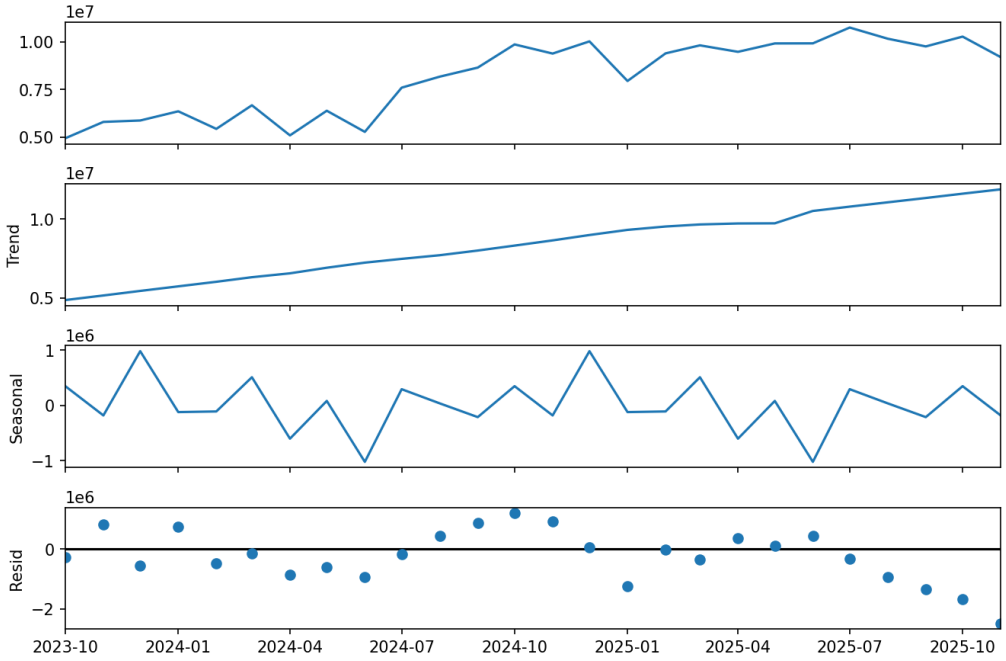
# Buoc 2: phan ra xu huong / mua vu / phan du (nhu Buoi 1)
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
result = seasonal_decompose(ser, model="additive", period=12)
result.plot()
plt.savefig("imgs/phan_ra_decompose.png", dpi=150, bbox_inches="tight
    ")
```



Kết quả thật

Chuỗi tăng trưởng 86% từ đầu kỳ đến cuối kỳ (khoảng 4,9 triệu lên 11,9 triệu chiếc/tháng) — một ngành hàng đang mở rộng mạnh, khác hẳn tốc độ tăng trưởng ổn định của SKU Nồi cơm điện dùng xuyên suốt khóa học. Đây chính là giá trị của việc luyện tập trên nhiều bộ dữ liệu: mỗi ngành hàng có "tính cách" khác nhau, và code tổng quát phải chạy tốt trên cả hai.



Phân rã (decompose) cho thấy rõ 3 thành phần: xu hướng (Trend) tăng gần như tuyến tính

suốt 2 năm, mùa vụ (Seasonal) dao động trong khoảng ±1 triệu chiếc quanh trend, và phần dư (Resid) không còn mẫu hình rõ rệt — đúng như lý thuyết Buổi 1 đã học.

Liên hệ với nguyên tắc Vibe Coding (Mục 9.1)

Ví dụ này minh hoạ trực tiếp nguyên tắc đã học: để tái sử dụng đoạn code phân rã chuỗi thời gian cho bộ dữ liệu Indonesia này, KHÔNG cần dán bất kỳ con số doanh thu thật nào vào cuộc trò chuyện với AI — chỉ cần biết cấu trúc dữ liệu đầu vào (2 cột: tháng và sản lượng) và mô tả mong muốn ("phân rã xu hướng/mùa vụ/phần dư"). Đây là cách áp dụng an toàn kỹ thuật đã học vào dữ liệu thật của công ty bạn sau này.

12 Đọc hiểu code mở rộng: Quadratic Trend

Phần này đọc một đoạn code NGẮN nhưng minh hoạ một nguyên tắc lập trình quan trọng: MỞ RỘNG một mô hình đã có bằng cách THÊM một biến, thay vì viết lại từ đầu.

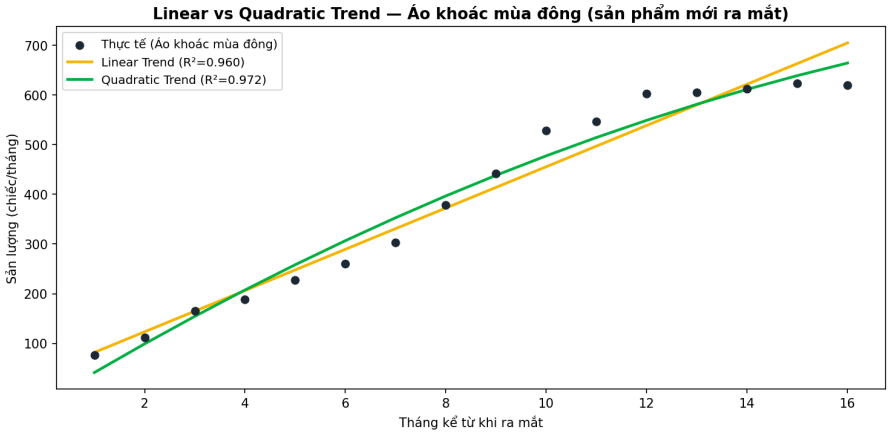
```
import statsmodels.api as sm

# Linear Trend - da hoc o Buoi 5
X_lin = sm.add_constant(t)
model_lin = sm.OLS(y, X_lin).fit()

# Quadratic Trend - CHI THEM 1 dong tao cot t^2, code con lai GIONG
    HET
X_quad = sm.add_constant(np.column_stack([t, t**2]))
model_quad = sm.OLS(y, X_quad).fit()

print(f"Linear R2={model_lin.rsquared:.3f}, Quadratic R2={model_quad.
    rsquared:.3f}")
```

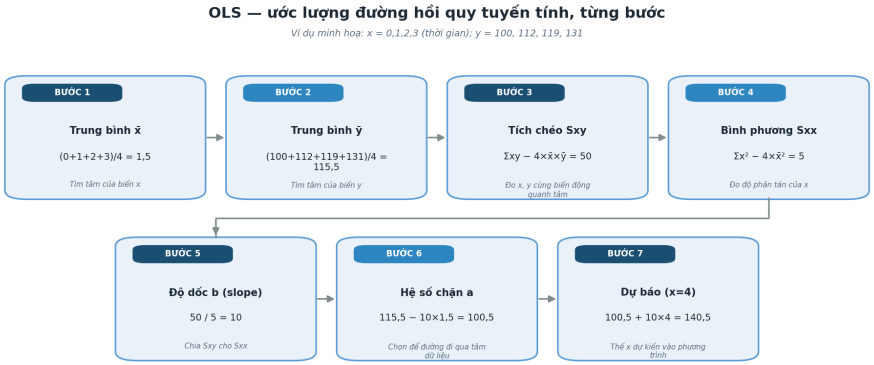
- **Dòng `np.column_stack([t, t**2])`:** ghép HAI mảng số thành một MA TRẬN hai cột — cột đầu là t, cột hai là t^2. Đây là cách NumPy tạo bảng nhiều biến giải thích, tương tự cách chọn nhiều cột liền kề khi dùng LINEST trong Excel.
- **So sánh với Linear:** CHỈ khác đúng phần `X_quad` thay vì `X_lin` — dòng `sm.OLS(y, X_...).fit()` giữ NGUYÊN không đổi. Đây chính là lý do Quadratic Trend không phải "kỹ thuật mới"mà là MỞ RỘNG trực tiếp của Regression đã học — cùng một hàm `sm.OLS`, chỉ khác số cột đưa vào.
- **Đọc hệ số:** `model_quad.params` trả về mảng ba giá trị theo thứ tự $[b_0, b_1, b_2]$ — LƯU Ý thứ tự này khớp với thứ tự cột đưa vào `column_stack`, không phải thứ tự ngược như hàm LINEST của Excel — một điểm dễ nhầm lẫn khi đối chiếu hai công cụ.



Kết quả thật

Trên Áo khoác mùa đông, Quadratic Trend giảm MSE 31,2% so với Linear — và dấu ÂM của hệ số b_2 (in ra bằng `model_quad.params[2]`) xác nhận đúng trực giác: tăng trưởng đang chững lại. Đây là cách nhanh để tự kiểm tra code chạy đúng hay sai — nếu dấu hệ số ngược với hình dạng đồ thị nhìn thấy, khả năng cao có lỗi ở đâu đó.

Dòng `model_lin = sm.OLS(y, X_lin).fit()` ở trên chính là lời gọi Python của quy trình OLS đơn biến đã học ở Buổi 5 — sơ đồ dưới đây ôn lại trình tự các bước tính tay của quy trình đó trên một ví dụ minh hoạ:



13 Đọc hiểu code mở rộng: hồi quy đa biến với yếu tố ngoại sinh liên tục

Phần Lecture Notes Chương 5 đã trình bày cách mở rộng mô hình Nồi cơm điện bằng hai biến liên tục (CSI, MetalIdx) và phát hiện đa cộng tuyến giữa `t` và `MetalIdx`. Đoạn code dưới đây là toàn bộ quy trình bốn bước đó viết bằng `statsmodels` — không dùng kỹ thuật mới nào ngoài `sm.OLS` đã quen thuộc, chỉ khác ở CÁCH TỔ CHỨC nhiều mô hình để so sánh có hệ thống.

```
import numpy as np, pandas as pd, statsmodels.api as sm
```

```
# df da co san cac cot: t, D2..D12 (dummy thang), Promo, CSI,
    MetalIdx, NhuCau
base_cols = ["t"] + [f"D{m}" for m in range(2, 13)]

def fit(cols):
    X = sm.add_constant(df[cols])
    return sm.OLS(df["NhuCau"], X).fit()

# Buoc 1: uoc luong nhieu mo hinh ung vien
m2 = fit(base_cols + ["Promo"])
m3 = fit(base_cols + ["Promo", "CSI"])
m5 = fit(base_cols + ["Promo", "CSI", "MetalIdx"])

# Buoc 2: kiem tra da cong tuyen bang ma tran tuong quan
print(df[["t", "CSI", "MetalIdx"]].corr())          # t va MetalIdx:
    r = 0.977
print(m5.pvalues[["CSI", "MetalIdx"]])                # MetalIdx: p ~
    0.49 (khong y nghia)

# Buoc 3: loai MetalIdx (da cong tuyen, khong y nghia) --> chon m3
# Buoc 4: kiem dinh ngoai mau bang train/test, KHONG chi nhin R2
    trong mau
train, test = df.iloc[:30], df.iloc[30:]

def oos_mse(cols):
    mdl = sm.OLS(train["NhuCau"], sm.add_constant(train[cols])).fit()
    X_test = sm.add_constant(test[cols], has_constant="add")[mdl.
        model.exog_names]
    pred = mdl.predict(X_test)
    return np.mean((test["NhuCau"].values - pred.values) ** 2)

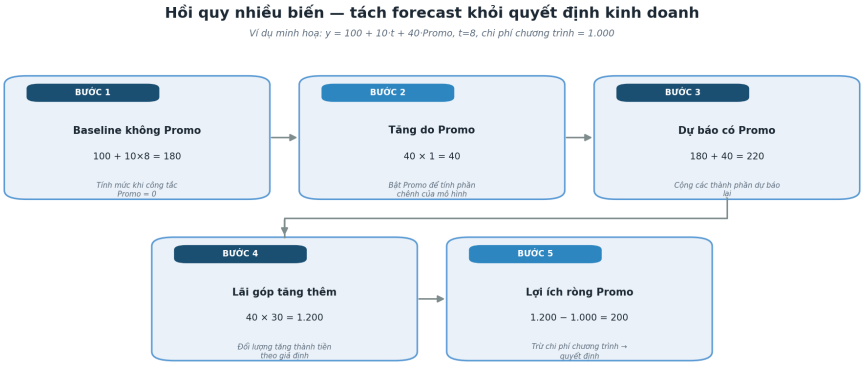
print("MSE ngoai mau - M2:", oos_mse(base_cols + ["Promo"]))
print("MSE ngoai mau - M3:", oos_mse(base_cols + ["Promo", "CSI"]))
    # THAP NHAT
print("MSE ngoai mau - M5:", oos_mse(base_cols + ["Promo", "CSI", "
    MetalIdx"]))
```

- **Hàm `fit(cols)`:** đóng gói lặp lại đúng hai dòng `sm.add_constant` + `sm.OLS(...).fit()` đã học — viết một lần, gọi nhiều lần với danh sách cột khác nhau, tránh copy-paste năm lần cho năm mô hình.
- **`df[[...]].corr()`:** trả về ma trận tương quan Pearson giữa các cột được chọn — đây chính là Panel A của biểu đồ trong Lecture Notes, chỉ khác là ở đây in ra dạng bảng số thay vì heatmap.
- **`m5.pvalues[[...]]`:** truy cập p-value của từng hệ số hồi quy theo TÊN cột (khác với `model_quad.params` ở phần Quadratic Trend truy cập theo VỊ TRÍ) — cách này an toàn hơn khi mô hình có nhiều biến, vì không phải nhớ thứ tự cột.
- **`has_constant="add"`** trong `sm.add_constant` khi áp dụng cho tập test: bắt buộc thêm cột hằng số ngay cả khi dữ liệu test (do biến đổi ngẫu nhiên) trông như không cần — nếu thiếu tham số này, đôi khi `statsmodels` sẽ KHÔNG thêm cột hằng số vào tập test, gây lệch số cột giữa X lúc `fit` và X lúc `predict`.

Vì sao không gộp bước 3 và 4 làm một

Học viên mới thường muốn "cứ thử hết các biến rồi xem MSE cái nào thấp nhất". Cách làm ở trên tách RÕ hai bước: trước tiên dùng lý luận thống kê (tương quan, p-value) để LOẠI bớt ứng viên rõ ràng có vấn đề, sau đó mới dùng train/test để xác nhận. Nếu chỉ dò MSE trên rất nhiều tổ hợp biến mà không có bước sàng lọc bằng lý luận, dễ rơi vào tình trạng chọn mô hình ăn may theo đúng tập test đang có — giống lỗi "nhìn trước đáp án" đã học ở Case Study Buổi 7.

Hàm `fit(cols)` ở trên tổng quát hoá cho NHIỀU biến giải thích cùng lúc — sơ đồ dưới đây ôn lại trình tự các bước của một hồi quy nhiều biến (Trend + Dummy mùa vụ + Promo) trên một bộ số liệu ví dụ đơn giản hoá, dùng để đối chiếu nhanh với các bước `fit`/`predict` trong code:



14 Đọc hiểu code nâng cao: hàm Croston cho nhu cầu rời rạc

Phần này đọc hiểu một đoạn code THẬT phức tạp hơn các ví dụ trước — hàm tính họ phương pháp Croston (dùng cho SKU có nhiều tháng nhu cầu bằng 0, xem lý thuyết đầy đủ ở Lecture Notes Chương 6). Đây là ví dụ tốt để ôn lại CẢ vòng lặp `for` (Mục 5) LẪN câu lệnh điều kiện `if/else`, kết hợp trong cùng một hàm — mẫu hình rất phổ biến khi code các công thức đệ quy (công thức kỳ sau phụ thuộc kỳ trước) như Exponential Smoothing, Holt-Winters, và Croston.

14.1 Code đầy đủ, đọc từng khối

```
def croston_family(series, alpha=0.2, beta=0.1, variant='croston'):
    n = len(series)
    # Buoc 1: tim vi tri KY DAU TIEN co phat sinh nhu cau (>0)
    nonzero_idx = [i for i, v in enumerate(series) if v > 0]
    forecasts = [None] * n   # khoi tao list ket qua, chua co gia tri

    if variant in ('croston', 'sba'):
        # Khoi tao z (quy mo), p (khoang cach), q (bo dem)
```

```
        z = series[nonzero_idx[0]]
        p = nonzero_idx[0] + 1
        q = 1
        for i in range(nonzero_idx[0] + 1, n):
            if series[i] > 0:                # CO nhu cau ky nay
                z = alpha * series[i] + (1 - alpha) * z
                p = alpha * q + (1 - alpha) * p
                q = 1                        # reset bo dem
            else:                            # KHONG co nhu cau ky
                nay
                q += 1                        # tang bo dem cho lan
                    sau
            # du bao SBA co he so (1-alpha/2), Croston goc thi khong
            forecasts[i] = (1 - alpha/2) * (z/p) if variant == 'sba'
                else z/p

    elif variant == 'tsb':
        z = series[nonzero_idx[0]]
        prob = 1.0 / (nonzero_idx[0] + 1)
        for i in range(nonzero_idx[0] + 1, n):
            if series[i] > 0:
                z = alpha * series[i] + (1 - alpha) * z
                prob = beta * 1 + (1 - beta) * prob   # CAP NHAT du
                    khi co nhu cau
            else:
                prob = beta * 0 + (1 - beta) * prob   # VA CA khi
                    khong co
            forecasts[i] = prob * z

    return forecasts
```

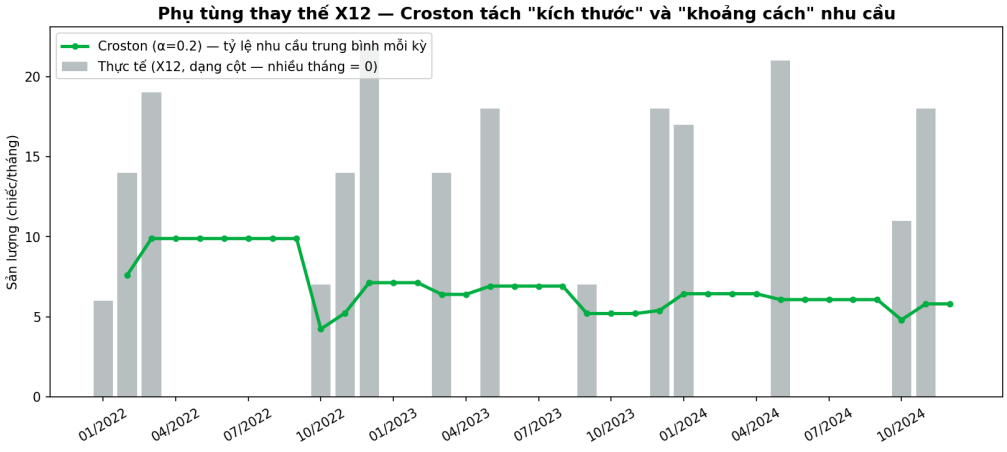
- **Dòng `nonzero_idx = [...]`:** đây là một *list comprehension* — cách viết gọn của Python để tạo một list mới bằng cách lọc từ list cũ. Đọc thành lời: "lấy chỉ số `i` của MỌI phần tử `v` trong `series`, với điều kiện `v > 0`". Kết quả là danh sách các vị trí có phát sinh nhu cầu.
- **Dòng `forecasts = [None] * n`:** tạo một list gồm `n` phần tử, tất cả đều là `None` (nghĩa là "chưa có giá trị") — kỹ thuật khởi tạo phổ biến khi kết quả sẽ được ĐIỀN DẦN vào bên trong vòng lặp, không phải tính ngay một lần.
- **Khối `if variant in ('croston', 'sba'):`:** toán tử `in` kiểm tra xem giá trị bên trái có nằm trong danh sách/tập hợp bên phải hay không — ở đây kiểm tra tham số `variant` có phải là `'croston'` HOẶC `'sba'` không, gộp chung xử lý cho cả hai vì công thức cập nhật `z` và `p` giống hệt nhau, chỉ khác công thức DỰ BÁO cuối cùng.
- **Vòng lặp `for i in range(...)`, bên trong có `if series[i] > 0: ... else: ...`:** đây là mẫu hình QUAN TRỌNG NHẤT cần hiểu — mỗi kỳ `i`, chương trình KIỂM TRA điều kiện, rồi chọn MỘT trong hai nhánh xử lý khác nhau. Nếu điều kiện đúng (có nhu cầu), chạy các dòng thụt lề dưới `if`; nếu sai, chạy các dòng thụt lề dưới `else`. Ký hiệu thụt lề (indentation) trong Python CHÍNH LÀ cách xác định dòng nào thuộc nhánh nào — đây là lỗi `IndentationError` phổ biến đã học ở Mục 8 nếu thụt lề sai.
- **Dòng `forecasts[i] = (1 - alpha/2) * (z/p) if variant == 'sba' else z/p`:** đây là cú pháp *if-else trên một dòng* (conditional expression) — một cách viết gọn của Python,

equivalent với việc viết `if variant=='sba':` rồi gán `forecasts[i]` theo công thức SBA, `else:` thì gán theo công thức Croston gốc — nhưng viết gọn trong MỘT dòng duy nhất. Cấu trúc chung: `giá_trị_nếu_đúng if điều_kiện else giá_trị_nếu_sai`.

Cách kiểm tra

Muốn chắc chắn đã hiểu đúng nhánh `if/else` nào chạy tại một kỳ cụ thể, chèn tạm một dòng `print(f"Kỳ {i}: series={series[i]}, nhánh={'CÓ' if series[i]>0 else 'KHÔNG'}")` ngay trong vòng lặp — đây là kỹ thuật "debug bằng print" rất hữu ích khi mới đọc code có nhiều nhánh điều kiện lồng nhau.

14.2 Kết quả thật trên SKU X12

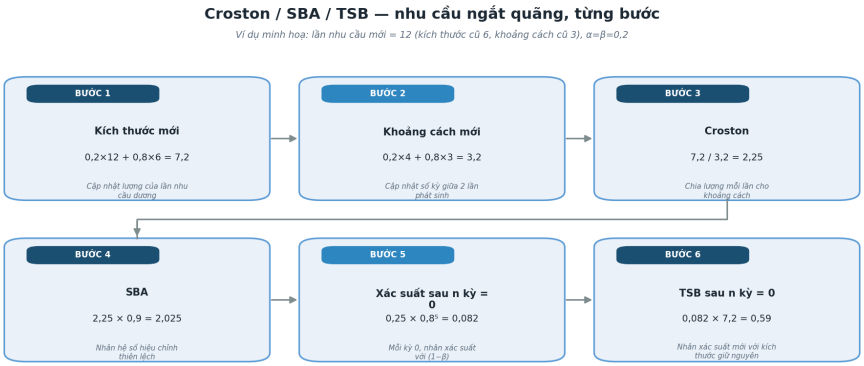


Chạy hàm trên với `variant='croston'`, `alpha=0.2` cho SKU Phụ tùng thay thế X12 (61,1% số tháng bằng 0) cho ra đường dự báo giữ ổn định suốt các giai đoạn im lặng, đúng như lý thuyết Chương 6 của Lecture Notes đã trình bày đầy đủ công thức và ba biến thể Croston/SBA/TSB.

Đối chiếu Excel và Python (Cách 3, Mục 8.2)

Workbook chính có sẵn sheet `Croston_XuLyRoiRac` với công thức Excel kéo-dãn tương đương — kết quả tại từng tháng PHẢI khớp chính xác giữa Excel và Python (đã kiểm chứng: 4,229 ở cả hai). Nếu không khớp, nhiều khả năng công thức Excel đang tham chiếu sai dòng — lỗi thường gặp khi dựng công thức đệ quy (mỗi dòng phụ thuộc dòng liền trước) bằng Excel.

Hàm `croston_family(...)` vừa đọc gộp CẢ ba biến thể Croston/SBA/TSB vào cùng một khối lệnh — sơ đồ dưới đây tách riêng trình tự cập nhật khi có một lần nhu cầu mới phát sinh, để đối chiếu từng dòng công thức với từng nhánh `if/else` trong code:



15 Đọc hiểu code kiểm tra thật: Train/Test Split

Phần cuối cùng đọc một đoạn code giải quyết một vấn đề CĂN BẢN: làm sao biết một mô hình dự báo THẬT SỰ chính xác, chứ không chỉ "khớp đẹp" với dữ liệu đã dùng để xây dựng nó? Đây là kỹ thuật **Train/Test Split** — xem lý thuyết đầy đủ ở Lecture Notes Chương 7.

15.1 Vì sao cần đọc kỹ đoạn code này

Mọi ví dụ code trước đó trong tài liệu này (SES ở Mục 4, Croston ở Mục 12) đều làm một việc: `fit` mô hình trên TOÀN BỘ dữ liệu, rồi tính sai số cũng trên chính dữ liệu đó. Đoạn code dưới đây khác ở MỘT điểm mấu chốt — dữ liệu được **cắt làm đôi TRƯỚC** khi làm bất kỳ điều gì khác:

```
# Buoc 1: CAT du lieu TRUOC KHI lam bat ky dieu gi khac
n_train = 30
y_train, y_test = y_full[:n_train], y_full[n_train:]

# Buoc 2: CHI fit tren train (moi ham .fit() duoi day deu nhan
    y_train,
# KHONG PHAI y_full)
ses_fc = SimpleExpSmoothing(y_train).fit().forecast(6)

# Buoc 3: so sanh du bao voi y_test - du lieu model CHUA TUNG thay
mape_test = mape(y_test, ses_fc)
```

- **Dòng `y_train, y_test = y_full[:n_train], y_full[n_train:]`:** đây là cú pháp *slicing* của Python đã quen thuộc — `[:n_train]` lấy TỪ ĐẦU đến trước vị trí `n_train`, `[n_train:]` lấy TỪ vị trí đó ĐẾN HẾT. Việc cắt này PHẢI xảy ra ở DÒNG ĐẦU TIÊN của toàn bộ quy trình — nếu vô tình `fit` trước rồi mới cắt, hoặc dùng `y_full` thay vì `y_train` ở bất kỳ bước nào, kết quả kiểm tra sẽ không còn đáng tin.
- **Dòng `SimpleExpSmoothing(y_train).fit()`:** SO SÁNH với cách gọi quen thuộc `SimpleExpSmoothing(series)` ở Mục 4 — CHỈ khác đúng MỘT biến đầu vào (`y_train` thay vì `series` chứa toàn bộ dữ liệu). Đây là điểm rất dễ đọc sót khi lướt nhanh qua code.

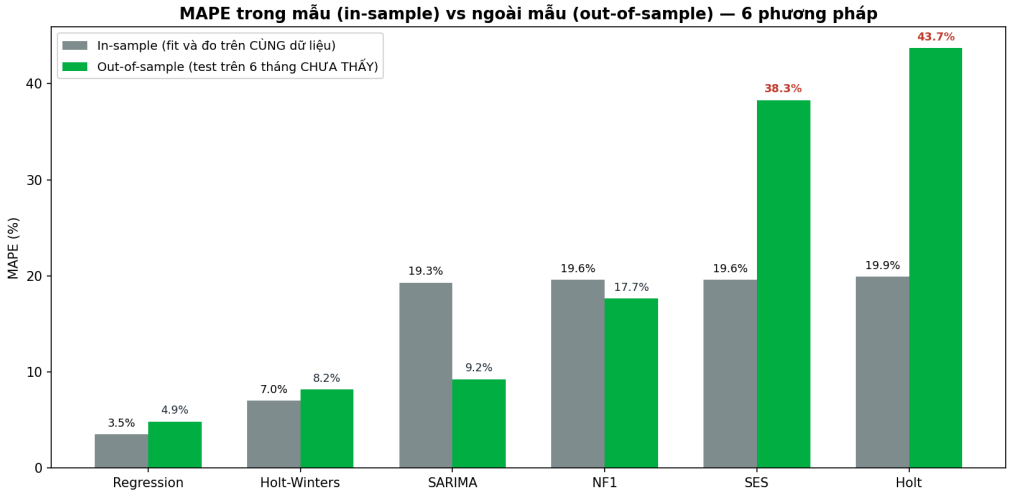
- **Hàm `mape(y_test, ses_fc)`:** đối số ĐẦU là dữ liệu THẬT (`y_test`), đối số THỨ HAI là dự báo (`ses_fc`) — LUÔN kiểm tra thứ tự đối số khi đọc code người khác viết, vì đảo ngược thứ tự không gây lỗi cú pháp (Python vẫn chạy được) nhưng cho kết quả TÍNH SAI hoàn toàn mà không có cảnh báo nào.

15.2 Điểm cần đặc biệt cẩn thận: rò rỉ dữ liệu khi chọn tham số ARIMA

```
# SAI - grid search AIC tren TOAN BO du lieu (bao gom ca test) roi
# moi chia train/test se lam RO RI thong tin tu test vao qua trinh
# chon tham so
best_order = grid_search_aic(y_full)    # KHONG duoc lam the nay

# DUNG - grid search AIC CHI tren train
best_order = grid_search_aic(y_train)  # dung cach nay
```

Đây là lỗi TINH VI, dễ mắc phải nhất khi tự viết code Train/Test Split: bước *chọn tham số* (không chỉ bước *ước lượng* tham số) cũng phải chỉ nhìn thấy `y_train`. Script `Buoi7_03_TrainTestSplit.py` thực hiện đúng nguyên tắc này — chạy grid search AIC hoàn toàn trên 30 tháng train trước khi đụng đến 6 tháng test.



Kết quả thật

Chạy đúng script trên cho thấy SES và Holt — vốn không tệ lắm trong mẫu (khoảng 20%) — sụp đổ hoàn toàn khi kiểm tra ngoài mẫu (38–44%), trong khi Holt-Winters và Regression giữ vững phong độ. Đây là bằng chứng thực nghiệm mạnh nhất cho nguyên tắc Vibe Coding: luôn KIỂM CHỨNG bằng dữ liệu thật, đừng chỉ tin vào một con số MAPE duy nhất.

16 Công cụ thực hành: Forecasting Lab (ứng dụng Streamlit)

Song song với các script `.py` chạy trong terminal đã học xuyên suốt tài liệu này, khóa học cung cấp thêm một ứng dụng web tương tác — **Forecasting Lab** — xây dựng bằng Streamlit, cho phép thực hành TOÀN BỘ quy trình forecast bằng cách kéo-thả và bấm nút, không cần gõ code, nhưng vẫn dùng ĐÚNG những công thức đã học. Ứng dụng gồm mười trang (tab), mỗi trang tương ứng một chương của tài liệu:

Trang	Nội dung, tương ứng chương nào
Tổng quan	Ma trận Pegels 9 ô, quy trình 6 bước (Chương 1, 10)
Phân loại hình dạng nhu cầu	CV, %kỳ=0, SLOPE (Chương 1)
Baseline: NF1, NF2	7 chỉ số sai số kể cả WMAPE (Chương 2)
Exponential Smoothing	SES, Holt, Holt-Winters (Chương 3)
ARIMA Lab	ADF, ACF/PACF, bản đồ nhiệt AIC, Ljung-Box (Chương 4)
Regression	Linear/Quadratic Trend, biến giả, Promo, yếu tố ngoại sinh liên tục + đa cộng tuyến (Chương 5)
Vận hành	Safety Stock, ROP, Tracking Signal, Croston/SBA/TSB (Chương 6)
Train/Test · Walk-Forward	Chia thời gian, kiểm tra quá khớp (Chương 7)
Outlier & Dữ liệu thiếu	IQR, Z-score, nội suy (Chương 8)
So sánh tổng hợp	Bảng xếp hạng MAPE tự động ghi nhận từ mọi trang đã chạy

Vì sao đáng đọc code của app, không chỉ bấm nút

Toàn bộ mười một file trong `src/` đều là những ví dụ THỰC TẾ, HOÀN CHỈNH của các kỹ thuật đã học trong tài liệu này — đóng gói hàm với `def`, kiểm tra đầu vào bằng `raise ValueError(...)` thay vì để chương trình dừng đột ngột (Mục 8), dùng `dataclass` để gói nhiều kết quả trả về thành MỘT đối tượng có tên rõ ràng thay vì một tuple khó nhớ thứ tự. Đọc qua `src/metrics.py` hoặc `src/naive_smoothing.py` là cách ôn lại nhanh toàn bộ Chương 2–3 dưới dạng code production-ready, không phải code demo một lần rồi bỏ.

Lưu ý khi tự mở rộng app

Nếu muốn thêm một trang mới (ví dụ thử một phương pháp ngoài chương trình), hãy viết phần TÍNH TOÁN thành hàm mới trong `src/` trước, kiểm chứng bằng dòng lệnh — giống mọi module khác trong app — RỒI mới viết phần giao diện Streamlit gọi đến hàm đó. Thứ tự này giúp gỡ lỗi dễ hơn nhiều so với viết cả tính toán lẫn giao diện trộn lẫn trong một file.

17 Những dòng code này đang chạy ở đâu trong phần mềm thương mại?

Một câu hỏi hợp lý sau khi gõ xong `ExponentialSmoothing(...).fit()` hay `ARIMA(...).fit()` bằng `statsmodels`: các hàm Python này có "đồ chơi cho người mới học", tách biệt với thế giới phần mềm doanh nghiệp thật, hay không?

Câu trả lời ngắn: KHÔNG. Đây chính xác là lớp thuật toán — Holt-Winters, ARIMA, hồi quy tuyến tính, Croston — chạy bên trong tầng *statistical baseline* của hầu hết nền tảng APS thương mại (SAP IBP, Kinaxis, ToolsGroup, và nhiều nền tảng khác), như *Lecture Notes* (mục "Dự báo Xác suất: khóa học này đang ở đâu so với phần mềm hàng đầu?", Chương 6) đã phân tích chi tiết: các nền tảng này được xếp vào nhóm "Semi-genuine" hoặc "Marketing-level" theo đánh giá của Lokad (12/2025) một phần chính vì lớp lõi thống kê của chúng vẫn là những công thức bạn vừa code tay trong tài liệu này, chỉ được đóng gói lại với giao diện kéo-thả và quy mô hàng chục nghìn SKU.

Điều Python/`statsmodels` trong tài liệu này CHƯA làm — và là ranh giới với nhóm phần mềm "Genuine" xác suất thực thụ (xem lại Lecture Notes) — là: xuất ra TOÀN BỘ phân phối xác suất của nhu cầu thay vì một con số điểm, và tối ưu hoá trực tiếp thành quyết định (đơn hàng/phân bổ) mà không quy về Safety Stock. Học viên muốn đi tiếp theo hướng này có thể tìm hiểu `statsmodels.regression.quantile_regression` (hồi quy phân vị — mở rộng trực tiếp từ Chương 5) hoặc thư viện `GluonTS`/`PyMC` như Lecture Notes đã gợi ý ở mục lộ trình học tiếp theo.

Code & Demo

Các bạn có thể truy cập Web demo thực hành và GitHub repository theo các liên kết dưới đây:

- **Web demo thực hành:** Forecasting Lab
- **GitHub repository:** Link GitHub